

WHAT WORKED, AND  
WHAT BROKE

# VIBE CODING FROM A SYSTEM SOFTWARE AND PERFORMANCE ENGINEERING PERSPECTIVE

**BRICE VIDEAU**

Argonne Leadership Computing  
Facility

Argonne National Laboratory  
ATPESC · August 2026



Argonne National Laboratory is a  
U.S. Department of Energy laboratory  
managed by UChicago Argonne, LLC.



**Argonne Leadership  
Computing Facility**

# I RAN THIS EXPERIMENT ONCE BEFORE

DOE's 1,000 Scientist AI Jam Session — February 28, 2025

- I brought the exact task this whole talk is about:
  - Modify rust-gpu to target OpenCL SPIR-V; offload a Rust vector-add kernel to an OpenCL device
- Fully specified: build-time target selection, OpenCL 2.1+, the opencl3 crate, a runner that validates its results
- Four frontier reasoning models: o1-deepresearch, o1-pro, o3-mini-high, Claude 3.7 Sonnet Extended
- My conclusion: unless you are an expert in the field, most of what it generates is misleading

# WHAT HAPPENED IN FEBRUARY 2025

The failure mode was not bad code. It was confident false reporting.

## Reported false work

It documented, in the README and developer guide, changes it had made to the backend. It had modified neither file.

## Invented the API

`create_program_from_spirv` does not exist in the `opencl3` crate. Buffers sized by element count rather than bytes — an immediate segfault.

## Only one even tried

Three returned a plan. The one that did edit the library hallucinated functions and defined methods outside their types.

## What changed by 2026

Models can now hold a port this size. What did not change: they still report success they cannot substantiate.

# TWO CASE STUDIES, ONE AXIS

The axis is not difficulty — it is whether I can verify the work

## CCS — familiar ground

- Autotuning middleware for HPC runtimes; an ECP continuation
- Production C99 that I wrote and still maintain
- I can read any diff and know if it is right
- My role: reviewer and taste-keeper

## rust-gpu + claspr — uncharted

- Graphics SPIR-V to OpenCL SPIR-V — Rust on Aurora
- I am an OpenCL expert; not a Rust or SPIR-V expert
- Nothing comparable exists in the training corpus
- My role: oracle designer



# CCS IN SIXTY SECONDS

## Autotuning configuration spaces, shared across languages

- C99 library: configuration spaces, objective spaces, tuners, ask/tell
- Numerical, categorical, ordinal, discrete and string parameters
- Conditions, forbidden clauses, expressions, feature contexts
- JSON and binary serialization; Python and Ruby bindings
- So a tuner written in one language can be driven from a runtime in another
- About 53,000 lines of C — and I know all of them

# RUST-GPU + CLASPR IN SIXTY SECONDS

## Rust on Intel GPUs, and a single-source programming model

- rust-gpu compiles Rust to SPIR-V — but targeted Vulkan only, so no Aurora
- We added the OpenCL Kernel execution model: Physical64 addressing, OpenCL.std intrinsics, native CL vector types
- claspr is the host layer: single-source proc-macros, typed launches, type-state buffer safety
- One async chain composes multi-stage GPU work into a single event graph

# PART 1

## WHAT ACTUALLY LANDED



U.S. DEPARTMENT  
of **ENERGY**

Argonne National Laboratory is a  
U.S. Department of Energy laboratory  
managed by UChicago Argonne, LLC.



**Argonne Leadership  
Computing Facility**



# WHAT LANDED

Six weeks on CCS; four months on rust-gpu, claspr and pocl

## CCS

- 99 pull requests, #24 through #122 — 96 merged, 3 closed
- Feb 26 → Apr 7, about 16 merged PRs per week
- Sanitizer and coverage jobs added to CI
- Every PR single-topic, read and merged by hand

## rust-gpu · claspr · pocl

- rust-gpu: +24,262 / -384 across 765 files; 8 OpenCL targets
- claspr: 466 commits, ~49K lines of Rust, 417 tests
- Green on three OpenCL runtimes: pocl, rusticl, Intel NEO
- pocl upstream: 4 PRs merged, 2 issues filed and fixed



# WHAT THE CORRECTNESS WORK LOOKED LIKE

Real defects, in both directions — found by tools, not by reading

## CCS — via UBSan and gcov

- Function-pointer type mismatches across the ops hierarchy — textbook C UB
- Null pointer passed to memcpy when the size is zero
- JSON integers silently losing precision past  $\pm 2^{53}$
- Six real bugs in the Ruby and Python bindings

## rust-gpu — via difftests

- A genuine miscompile: OpIAdd with mismatched operand widths
- Signedness stripped at emission, kept internally for dispatch
- spirv-opt segfaults on valid SPIR-V — now run in a forked child
- Kernel argument order preserved via a side-channel decoration

# UPSTREAM IN SOMEONE ELSE'S RUNTIME

Four merged pool PRs — and the review that caught the agent

## What got merged

- A command whose dependency had already failed ran on freed memory
- Concurrent double-finish of one event, from two threads
- A spec-mandated retain missing across a synchronous callback
- Both races found by our own workloads, minimized, filed, merged

## What the maintainer caught

- Changes requested — seven inline comments, none of them logic errors
- All seven were naming, redundancy or misplaced checks: taste
- Two flagged the agent's own comments contradicting each other
- It could not see that. A human reviewer could, in one pass.

# PART 2

## HOW THE HUMAN HAS TO WORK



U.S. DEPARTMENT  
of ENERGY

Argonne National Laboratory is a  
U.S. Department of Energy laboratory  
managed by UChicago Argonne, LLC.



Argonne Leadership  
Computing Facility



# DIRECTING VERSUS DIALOGUE

Same tool, same model — two completely different jobs for me

## Directing (CCS)

- Branch per task, pull request per task, single topic each
- I am the reviewer: every PR read and merged by hand
- Rebase on upstream after every merge
- Short PRs keep review cost below the value delivered

## Dialogue (rust-gpu + claspr)

- Start from a reproducer, not a specification
- Argue the design first — I bring the OpenCL semantics
- Spike first, land second; the spike is the design document
- I cannot check the code, so I check the behaviour



# HOW I VERIFIED

When you cannot read the diff, the oracle has to move outside you

## Familiar: verify by reading

- I read the diff, and I know the idioms it should have used
- Backed by valgrind, sanitizers, coverage and distcheck
- Review is cheap because my knowledge is the oracle
- Risk: habits leak past me while I read for correctness

## Uncharted: verify by construction

- Three OpenCL runtimes as a differential oracle — they must agree
- Four-way difftests: host and device, Rust and OpenCL, byte-for-byte
- CPU golden references, checked bit-exact
- Type-state buffers plus 24 compile-fail fixtures

# VERIFICATION AS ARCHITECTURE

Four things that caught real bugs the agent could not see

## Differential runtimes

Run on pool, rusticl and Intel NEO. They disagree — and the disagreement is the test.

## Sanitizers and tracers

UBSan found the C undefined behaviour. An API tracer found 16 context allocations and 0 releases.

## Bisect and repetition

Six runs per commit to find a race. 40/40, 60/60, bit-exact 5/5 to believe it is fixed.

## Compile-time gates

Type-state markers and golden-stderr fixtures. Invariants the agent cannot quietly regress.

# MAKING THE ORACLE STRUCTURAL

One Rust function: the kernel runs it, the host validates with it

## Code

```
#[claspr::device]
mod gpu {
    /// Pure Rust – called from the kernel AND from the host.
    pub fn collatz(mut n: u32) -> Option<u32> {
        let mut i = 0;
        while n != 1 { n = if n%2 == 0 {n/2} else {3*n+1}; i += 1; }
        Some(i)
    }

    #[claspr::kernel]
    pub fn collatz_kernel(
        #[spirv(global_invocation_id)] id: glam::USizeVec3,
        #[spirv(cross_workgroup)] data: &mut [u32],
    ) { data[id.x] = collatz(data[id.x]).unwrap_or(u32::MAX); }
}

let dev = DeviceSlice::from_slice(&ctx, &h)?;
let d = kernels.collatz_kernel([N], dev).wait()?;
d.read(&mut h).wait()?;

// Validate device output against the SAME function on the host.
let ok = (1..=N as u32).zip(&h)
    .all(|(i, &n)| n == gpu::collatz(i).unwrap_or(u32::MAX));
assert!(ok);
```

## Why this matters here

- One function, two compilers: rust-gpu to SPIR-V, cargo to the host
- No FFI mock, no hand-written reference to drift out of sync
- The validator is the implementation — it cannot disagree by accident
- Build the oracle in once, instead of trusting every change





# PART 3 WHERE IT BROKE, AND WHAT TO DO ABOUT IT



U.S. DEPARTMENT  
of **ENERGY**

Argonne National Laboratory is a  
U.S. Department of Energy laboratory  
managed by UChicago Argonne, LLC.

Argonne  
NATIONAL LABORATORY



Argonne Leadership  
Computing Facility

# PITFALLS DIFFER BY TERRAIN

Familiar code fails quietly; uncharted code fails late

## On familiar code

- Silently swapped clang-format for clang-format-17 — CI stayed green
- Defaults to laziness on cleanup paths and overflow checks
- Forgets project idioms after every compaction
  - Mitigation: short PRs and real diff review

## On uncharted code

- Context exhaustion: eight auto-compactions in one session
- “Cleaned up” spike code that was intentional design documentation
- An edit anchor silently deleted a CI step
  - Mitigation: end sessions early, pin toolchains

# THE BUG A FRIENDLY RUNTIME HIDES

## Your development box's forgiveness is a liability

- claspr kept a buffer permanently mapped and also passed it to kernels  
— Straight undefined behaviour per the OpenCL spec — no escape hatch
- pocl is permissive, so it worked. For weeks. Every test green.
- rusticl is correct, so it failed: host wrote 6, kernel sees 0
- pocl is the outlier here, not rusticl — the forgiving runtime taught us wrong
- A second implementation is not redundancy. It is why we found this at all.



# THE AGENT MISDIAGNOSED ITS OWN BUG

**Its narrative was confident and wrong; the tools were right**

- A simulation diverged. The agent declared the race pre-existing.
- Bisect, six runs per commit: green at a049ad4, first red at 236d9c0 — Its own commit — and the “clean baseline” it cited was inside the work it was defending
- A CPU golden broke the symmetry: equality cannot say which side is wrong
- The API trace gave the one-line proof: wait=NONE became wait=[3]
- Fixed — then bit-exact, five runs out of five



# CONTEXT IS A RESOURCE — SO OPTIMIZE IT

A performance-engineering reflex, turned on the agent itself

## The observation

- “A sub-agent burns ~200K tokens before it can make ANY change”
- The architecture had grown too complex to hold in a context window
- In 2025 I blamed the models partly on rust-gpu being spaghetti
- In 2026 I stopped calling that an excuse and measured it

## The optimization

- Profiled with rust-code-analysis to rank functions by complexity
- Worst function: cognitive 39 → 7
- Largest module: 11,023 → 3,674 lines, down 67%
- Behaviour-preserving, verified bit-identical on three runtimes

# WHAT ACTUALLY WORKS

Four gates, each earned by getting it wrong first

## Diff review is the gate

Not CI-green. Toolchain swaps, scope creep and dependency drift all pass CI comfortably.

## Pin your toolchain

Formatter version, runtime build, which driver actually loads. Verify what is loaded, not what is installed.

## Get a second opinion

A second implementation, a sanitizer, a tracer, a bisect. Never the agent re-reading its own work.

## End the session

Every compaction costs quality. Long rolling sessions feel productive and quietly degrade.

# TAKEAWAY

## You cannot let an agent run blindly and trust its work

- It produced real systems software — merged data-race fixes in an OpenCL runtime
- It also shipped spec-UB for weeks and misdiagnosed its own race
- Every bug here was caught by a sanitizer, a tracer, a bisect, a second implementation, or a human
  - Never once by the agent re-reading its own code
- Good news: you are spending two weeks learning exactly that toolbox
- What stays human: choosing the oracle, and reading the spec adversarially





# Argonne Leadership Computing Facility